

Compilation Project

ABBASSI, BOUZID

March 2026

Partie 7 – A simple stack language

Exercise 1 (expl)

A stack is a data structure following the *LIFO* principle (*Last In, First Out*): the last element pushed onto the stack is the first one to be removed.

The usual operations on a stack are:

- **push**: add an element on the top of the stack.
- **pop**: remove the top element of the stack.
- **top** (or **peek**): read the top element without removing it.
- **is_empty**: test whether the stack is empty.

Exercise 2 (expl)

Program: 0 push 12 push 7 swap sub

Parameters: $\emptyset$

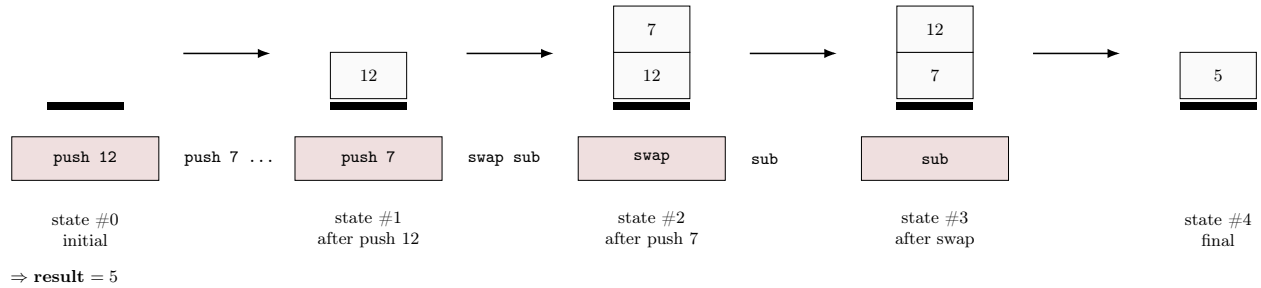


Figure 1: Detailed execution of 0 push 12 push 7 swap sub.

Exercise 3

Question 3.1 (expl)

The semantics of programs is defined by three rules.

1. If the program expects i arguments but receives n arguments with $i \neq n$, then execution immediately fails and returns **Err**.
2. If the number of arguments is correct, then the arguments are first placed on the initial stack, and the instruction sequence Q is executed. If this execution leads to a runtime error, then the whole program returns **Err**.

3. If the number of arguments is correct and the execution terminates normally on a non-empty stack of the form $v :: S$, then the result of the program is the top value v .

In short:

- wrong number of arguments $\Rightarrow \mathbf{Err}$,
- runtime error during execution $\Rightarrow \mathbf{Err}$,
- successful execution with a non-empty final stack $\Rightarrow$ result is the top of the stack.

Question 3.2 (math)

A case is missing: when the number of arguments is correct, execution terminates normally, but the final stack is empty. In that case, there is no result to return, so the program must return $\mathbf{Err}$.

The missing rule is:

$$\frac{Q, v_1 :: \dots :: v_n :: \emptyset \rightarrow^* \emptyset, \emptyset}{v_1, \dots, v_n \vdash n, Q \Rightarrow \mathbf{Err}}$$

Question 3.3 (math)

We now define the small-step semantics of instruction sequences.

Push

$$\overline{\text{push } k.Q, S \rightarrow Q, k :: S}$$

Pop

$$\overline{\text{pop}.Q, k :: S \rightarrow Q, S} \quad \overline{\text{pop}.Q, \emptyset \rightarrow \mathbf{Err}}$$

Swap

$$\overline{\text{swap}.Q, k_1 :: k_2 :: S \rightarrow Q, k_2 :: k_1 :: S}$$

$$\overline{\text{swap}.Q, \emptyset \rightarrow \mathbf{Err}} \quad \overline{\text{swap}.Q, k :: \emptyset \rightarrow \mathbf{Err}}$$

Add

$$\overline{\text{add}.Q, k_1 :: k_2 :: S \rightarrow Q, (k_1 + k_2) :: S}$$

$$\overline{\text{add}.Q, \emptyset \rightarrow \mathbf{Err}} \quad \overline{\text{add}.Q, k :: \emptyset \rightarrow \mathbf{Err}}$$

Sub

$$\overline{\text{sub}.Q, k_1 :: k_2 :: S \rightarrow Q, (k_1 - k_2) :: S}$$

$$\overline{\text{sub}.Q, \emptyset \rightarrow \mathbf{Err}} \quad \overline{\text{sub}.Q, k :: \emptyset \rightarrow \mathbf{Err}}$$

Mul

$$\overline{\text{mul}.Q, k_1 :: k_2 :: S \rightarrow Q, (k_1 \times k_2) :: S}$$

$$\overline{\text{mul}.Q, \emptyset \rightarrow \mathbf{Err}} \quad \overline{\text{mul}.Q, k :: \emptyset \rightarrow \mathbf{Err}}$$

Div

$$\frac{k_2 \neq 0}{\overline{\text{div}.Q, k_1 :: k_2 :: S \rightarrow Q, (k_1 \div k_2) :: S}}$$

$$\overline{\text{div}.Q, \emptyset \rightarrow \mathbf{Err}} \quad \overline{\text{div}.Q, k :: \emptyset \rightarrow \mathbf{Err}} \quad \overline{\text{div}.Q, k_1 :: 0 :: S \rightarrow \mathbf{Err}}$$

Rem

$$\frac{k_2 \neq 0}{\text{rem}.Q, k_1 :: k_2 :: S \rightarrow Q, (k_1 \bmod k_2) :: S}$$

$$\frac{}{\text{rem}.Q, \emptyset \rightarrow \text{Err}} \quad \frac{}{\text{rem}.Q, k :: \emptyset \rightarrow \text{Err}} \quad \frac{}{\text{rem}.Q, k_1 :: 0 :: S \rightarrow \text{Err}}$$

Final configuration When the instruction sequence is empty, execution is finished:

$$\emptyset, S$$

is a final configuration.

Exercise 5

Question 5.1

At this stage, variables cannot be compiled because the current version of *Pfx* does not provide any instruction to access an environment. Therefore, the compilation schema is defined only for variable-free arithmetic expressions.

Instruction sequences are built with the constructor “.” and the empty sequence is written $\emptyset$. We define the compilation as:

$$\mathcal{C}(e) = (0, \mathcal{G}(e, \emptyset))$$

where $\mathcal{G}(e, Q)$ means “generate the code that evaluates e and then continues with the instruction sequence Q ”.

We use the following correspondence between **Expr** binary operators and **Pfx** instructions:

$$\hat{+} = \text{add}, \quad \hat{-} = \text{sub}, \quad \hat{*} = \text{mul}, \quad \hat{/} = \text{div}, \quad \hat{\%} = \text{rem}.$$

The translation rules are:

$$\begin{aligned} \mathcal{G}(n, Q) &= \text{push } n.Q \\ \mathcal{G}(-e, Q) &= \mathcal{G}(e, \text{push } 0.\text{sub}.Q) \\ \mathcal{G}(e_1 \oplus e_2, Q) &= \mathcal{G}(e_2, \mathcal{G}(e_1, \hat{\oplus}.Q)) \quad (\oplus \in \{+, -, *, /, \%\}) \end{aligned}$$

Variables are not compiled at this stage: $\mathcal{G}(x, Q)$ is undefined because the current *Pfx* machine has no instruction to fetch the value of a variable.

The order in the binary-operation rule is important. In *Pfx*, arithmetic instructions use the top element of the stack as their first argument and the second element as their second argument. Therefore, to compile $e_1 - e_2$, we must first push the value of e_2 and then the value of e_1 , so that **sub** computes $e_1 - e_2$.

The intended correctness property is the following: if e is a variable-free expression whose value is v , then executing $\mathcal{G}(e, Q)$ on a stack S has the same effect as executing Q on the stack $v :: S$. In particular, executing $\mathcal{C}(e)$ leaves the value of e on top of the initial empty stack.

Exercise 9

Question 9.1

We do not need to *change the behavior* of the already defined *Pfx* instructions, but we do need to enlarge the domain on which they operate.

Previously, the stack contained only integers. After the introduction of executable sequences, a stack element becomes either:

$$v ::= n \mid \langle Q \rangle$$

where $n \in \mathbb{Z}$ and Q is an instruction sequence.

Therefore:

- the rules for **push**, **pop**, and **swap** keep the same operational behavior; they now manipulate stack *objects* instead of integers only;
- the arithmetic instructions **add**, **sub**, **mul**, **div**, and **rem** keep exactly the same computation rule, but they are only defined when their operands are integers;
- if one of these arithmetic instructions is applied to an executable sequence, the machine should raise a runtime error.

So the right answer is: the old rules are essentially preserved, but the type of stack elements must be generalized from integers to stack values $v \in \mathbb{Z} \cup \{\langle Q \rangle\}$.

Question 9.2

We give the operational semantics of the three new *Pfx* constructions: executable sequences, **exec**, and **get**. We write machine states as pairs (Q, S) where:

- Q is the currently executing instruction sequence;
- S is the stack;
- instruction sequences are written with the paper notation $I.Q$ and $\emptyset$.

When a whole sequence must be put in front of another one, we still use the same dot notation recursively:

$$\emptyset.Q = Q \quad \text{and} \quad (I.Q_1).Q_2 = I.(Q_1.Q_2)$$

Executable sequence. When an executable sequence is encountered in the current code, it is not executed immediately. It is pushed onto the stack as a value.

$$\overline{(\langle Q_1 \rangle.Q_2, S)} \longrightarrow \overline{(Q_2, \langle Q_1 \rangle :: S)}$$

exec. The instruction **exec** pops the top of the stack. This top element must be an executable sequence. The popped code is then executed immediately by being placed in front of the remaining current code.

$$\overline{(\text{exec}.Q, \langle Q' \rangle :: S)} \longrightarrow \overline{(Q'.Q, S)}$$

If the top element of the stack is not an executable sequence, the machine raises a runtime error.

get. The instruction **get** pops an integer i from the top of the stack, then copies onto the top of the stack the element of depth i in the remaining stack. The indexing is 0-based: depth 0 means the top of the remaining stack.

$$\overline{(\text{get}.Q, i :: v_0 :: v_1 :: \dots :: v_i :: S)} \xrightarrow{i \geq 0} \overline{(Q, v_i :: v_0 :: v_1 :: \dots :: v_i :: S)}$$

In all other cases, **get** raises a runtime error:

- if the top of the stack is not an integer;
- if the stack does not contain enough elements below this integer.

Remark. These rules are exactly what is needed for compiling simple functions:

- a function is translated into an executable sequence;
- an application uses **exec** to launch this sequence;
- the instruction **get** gives access to a parameter stored deeper in the stack.

Exercise 10

Question 10.1

We want to compile the expression

$$(\lambda x. x + 1) 2$$

into Pfx .

Using the translation principle of the statement:

- a function is translated into an executable sequence;
- an application first computes the argument, then the function value, then runs `exec`;
- after the call terminates, the argument is removed from the stack with `swap.pop`.

The compiled program is therefore:

$$(0, \text{push } 2.(\text{push } 1.\text{push } 1.\text{get}.\text{add}.\emptyset).\text{exec}.\text{swap}.\text{pop}.\emptyset)$$

Why is the function body compiled this way? Inside the body $x + 1$, the constant 1 is computed first. Once this value is pushed onto the stack, the parameter x is no longer at depth 0 but at depth 1. Therefore, the occurrence of x is compiled as:

$$\text{push } 1.\text{get}$$

Hence:

$$\lambda x. x + 1 \rightsquigarrow (\text{push } 1.\text{push } 1.\text{get}.\text{add}.\emptyset)$$

Step-by-step evaluation. We start from the empty stack.

$$(\text{push } 2.(Q_f).\text{exec}.\text{swap}.\text{pop}.\emptyset, [])$$

where

$$Q_f = \text{push } 1.\text{push } 1.\text{get}.\text{add}.\emptyset$$

$$\begin{aligned} (\text{push } 2.(Q_f).\text{exec}.\text{swap}.\text{pop}.\emptyset, []) &\rightarrow ((Q_f).\text{exec}.\text{swap}.\text{pop}.\emptyset, [2]) \\ &\rightarrow (\text{exec}.\text{swap}.\text{pop}.\emptyset, [Q_f; 2]) \\ &\rightarrow (Q_f.\text{swap}.\text{pop}.\emptyset, [2]) \\ &\rightarrow (\text{push } 1.\text{get}.\text{add}.\text{swap}.\text{pop}.\emptyset, [1; 2]) \\ &\rightarrow (\text{get}.\text{add}.\text{swap}.\text{pop}.\emptyset, [1; 1; 2]) \\ &\rightarrow (\text{add}.\text{swap}.\text{pop}.\emptyset, [2; 1; 2]) \\ &\rightarrow (\text{swap}.\text{pop}.\emptyset, [3; 2]) \\ &\rightarrow (\text{pop}.\emptyset, [2; 3]) \\ &\rightarrow (\emptyset, [3]) \end{aligned}$$

So the final result is:

$$(\lambda x. x + 1) 2 = 3$$

This exactly matches Figure 1 of the statement:

- first the argument 2 is pushed;
- then the executable sequence representing the function is pushed;
- `exec` starts the call with the argument still on the stack;
- the body computes the result while the parameter remains accessible;
- finally `swap.pop` removes the parameter and keeps only the result.

Question 10.2

We define a compilation function with an environment P that associates each accessible variable with its depth in the current stack.

$$\mathcal{G}_P(e, Q)$$

means: compile the expression e in environment P , assuming that after evaluation the generated code continues with instruction sequence Q .

Auxiliary definitions. When a value is pushed on the stack, all accessible variables become one position deeper. We write:

$$P^{+1}(x) = P(x) + 1$$

When introducing a new parameter y at the top of the stack, we extend the environment by:

$$P[y](x) = \begin{cases} 0 & \text{if } x = y, \\ P(x) + 1 & \text{if } x \neq y. \end{cases}$$

We also use the correspondence:

$$\hat{+} = \text{add}, \quad \hat{-} = \text{sub}, \quad \hat{*} = \text{mul}, \quad \hat{/} = \text{div}, \quad \hat{\%} = \text{rem}.$$

Compilation rules.

$$\mathcal{G}_P(n, Q) = \text{push } n.Q$$

$$\mathcal{G}_P(x, Q) = \text{push } P(x).\text{get}.Q$$

$$\mathcal{G}_P(-e, Q) = \mathcal{G}_P(e, \text{push } 0.\text{sub}.Q)$$

$$\mathcal{G}_P(e_1 \oplus e_2, Q) = \mathcal{G}_P(e_2, \mathcal{G}_{P^{+1}}(e_1, \hat{\oplus}.Q)) \quad (\oplus \in \{+, -, *, /, \%\})$$

$$\mathcal{G}_P(\lambda x. e, Q) = (\mathcal{G}_{P[x]}(e, \emptyset)).Q$$

$$\mathcal{G}_P(e_1 \ e_2, Q) = \mathcal{G}_P(e_2, \mathcal{G}_{P^{+1}}(e_1, \text{exec.swap.pop}.Q))$$

Explanation of the application rule. The expression e_2 is compiled first, so its value is pushed onto the stack. Therefore, while compiling e_1 , all previously accessible variables are one level deeper, which explains the use of P^{+1} .

After both subexpressions have been evaluated, the stack has the form:

[function code; argument; rest of the stack]

Then:

- **exec** starts the executable sequence;
- the function body computes a result while keeping the argument on the stack;
- **swap.pop** removes the argument and leaves only the result.

Exercise 10.4

We study the expression

$$((\lambda x. \lambda y. (x - y)) \ 12) \ 8$$

with the compilation scheme of Question 10.3.

Compiled Version

The generated *Pfx* program is:

```
(0, push 8.push 12.((push 0.get.push 2.get.sub.().).exec.swap.pop.exec.swap.pop.().))
```

The important point is that the outer function is compiled into an executable sequence whose body only pushes another executable sequence. That inner executable sequence contains the code for the body $x - y$.

Step-by-Step Evaluation

Let

$$Q_{inner} = \text{push } 0.\text{get}.\text{push } 2.\text{get}.\text{sub}.\emptyset$$

and

$$Q_{outer} = (Q_{inner}).\emptyset$$

We start from the empty stack:

$$(\text{push } 8.\text{push } 12.(Q_{outer}).\text{exec.swap.pop.exec.swap.pop}.\emptyset, [])$$
$$\begin{aligned} & (\text{push } 8.\text{push } 12.(Q_{outer}).\text{exec.swap.pop.exec.swap.pop}.\emptyset, []) \\ \rightarrow & (\text{push } 12.(Q_{outer}).\text{exec.swap.pop.exec.swap.pop}.\emptyset, [8]) \\ \rightarrow & ((Q_{outer}).\text{exec.swap.pop.exec.swap.pop}.\emptyset, [12; 8]) \\ \rightarrow & (\text{exec.swap.pop.exec.swap.pop}.\emptyset, [Q_{outer}; 12; 8]) \\ \rightarrow & (Q_{outer}.\text{swap.pop.exec.swap.pop}.\emptyset, [12; 8]) \\ \rightarrow & (\text{swap.pop.exec.swap.pop}.\emptyset, [Q_{inner}; 12; 8]) \\ \rightarrow & (\text{pop.exec.swap.pop}.\emptyset, [12; Q_{inner}; 8]) \\ \rightarrow & (\text{exec.swap.pop}.\emptyset, [Q_{inner}; 8]) \\ \rightarrow & (Q_{inner}.\text{swap.pop}.\emptyset, [8]) \\ \rightarrow & (\text{get.push } 2.\text{get.sub.swap.pop}.\emptyset, [0; 8]) \\ \rightarrow & (\text{push } 2.\text{get.sub.swap.pop}.\emptyset, [8; 8]) \\ \rightarrow & (\text{get.sub.swap.pop}.\emptyset, [2; 8; 8]) \end{aligned}$$

At this point, **get** tries to copy the element of depth 2 in the remaining stack. But the remaining stack is only:

$$[8; 8]$$

so there is no element at depth 2. The machine therefore raises a runtime error.

Interpretation

The result is wrong: we do not obtain $12 - 8 = 4$. Instead, evaluation fails.

What is happening is the following:

- after the first application, the result is only an executable sequence;
- this executable sequence is *not* a closure: it does not store the value of $x = 12$;
- the compilation of the inner body assumes that x will still be reachable in the stack at depth 2 when the second call starts;
- but the first application ends with **swap.pop**, which removes the argument 12 from the stack;
- therefore, when the inner function is later applied to 8, the variable x has disappeared.

So the bug is that executable sequences do not capture their environment. They only store code.

Exercise 11

Question 11.1

The new construction

$$\text{let } x = e_1 \text{ in } e_2$$

does not require a new constructor in the abstract syntax of *Expr*. It is only syntactic sugar.

Its translation into the already existing syntax is:

$$\text{let } x = e_1 \text{ in } e_2 \quad \equiv \quad (\lambda x. e_2) e_1$$

In the project abstract syntax, this means:

$$\text{App}(\text{Fun}(x, e2), e1)$$

Explanation. The idea is the same as in functional languages and in the λ -calculus:

- first compute the value of e_1 ;
- bind this value to the variable x ;
- then evaluate e_2 in this extended environment.

This behavior is exactly what function application already provides. The function

$$\lambda x. e_2$$

expects one argument, and applying it to e_1 gives the same meaning as

$$\text{let } x = e_1 \text{ in } e_2.$$

Therefore, to support **let**, it is enough to modify the parser so that this surface syntax is translated to $\text{App}(\text{Fun}(x, e2), e1)$.

Exercise 12

We want to compute, with the closure semantics of Figure 2, the value of

$$((\lambda x. \lambda y. (x - y)) 12) 8$$

in the empty environment.

Auxiliary Closures

Let

$$c_1 = \{\emptyset, x, \lambda y. (x - y)\}$$

and

$$c_2 = \{\{x \mapsto 12\}, y, x - y\}.$$

The first closure captures no variable, because the abstraction

$$\lambda x. \lambda y. (x - y)$$

has no free variable. The second closure captures the value of x , because after the first application the inner function

$$\lambda y. (x - y)$$

has x as a free variable.

Main Derivation

We first derive the value of the first application:

$$\frac{\emptyset \vdash \lambda x. \lambda y. (x - y) \Rightarrow c_1 \quad \emptyset \vdash 12 \Rightarrow 12 \quad \{x \mapsto 12\} \vdash \lambda y. (x - y) \Rightarrow c_2}{\emptyset \vdash (\lambda x. \lambda y. (x - y)) 12 \Rightarrow c_2} (\text{APP})$$

Then we derive the body of the second application:

$$\frac{\{x \mapsto 12, y \mapsto 8\} \vdash x \Rightarrow 12 \quad \{x \mapsto 12, y \mapsto 8\} \vdash y \Rightarrow 8}{\{x \mapsto 12, y \mapsto 8\} \vdash x - y \Rightarrow 4} (\text{BINOP})$$

Finally, we derive the whole term:

$$\frac{\emptyset \vdash (\lambda x. \lambda y. (x - y)) 12 \Rightarrow c_2 \quad \emptyset \vdash 8 \Rightarrow 8 \quad \{x \mapsto 12, y \mapsto 8\} \vdash x - y \Rightarrow 4}{\emptyset \vdash ((\lambda x. \lambda y. (x - y)) 12) 8 \Rightarrow 4} (\text{APP})$$

Justification of the Premises

The three non-trivial premises are justified as follows.

1. Evaluation of the outer function. By rule FUN,

$$\emptyset \vdash \lambda x. \lambda y. (x - y) \Rightarrow \{\emptyset, x, \lambda y. (x - y)\}$$

because the abstraction has no free variable to capture.

2. Evaluation of the inner function after the first application. After the first application, the body is evaluated in the environment

$$\{x \mapsto 12\}.$$

Therefore, again by rule FUN,

$$\{x \mapsto 12\} \vdash \lambda y. (x - y) \Rightarrow \{\{x \mapsto 12\}, y, x - y\}.$$

3. Evaluation of the arithmetic body. By two uses of rule VAR,

$$\{x \mapsto 12, y \mapsto 8\} \vdash x \Rightarrow 12 \quad \text{and} \quad \{x \mapsto 12, y \mapsto 8\} \vdash y \Rightarrow 8.$$

Then by rule BINOP,

$$\{x \mapsto 12, y \mapsto 8\} \vdash x - y \Rightarrow 12 - 8 = 4.$$

Conclusion

Hence the proof derivation concludes that

$$\emptyset \vdash ((\lambda x. \lambda y. (x - y)) 12) 8 \Rightarrow 4.$$

So, with closures, the term of Question 10.4 evaluates correctly to 4.

Exercise 13

Question 13.1

With the current version of *Pfx* from the previous section, it is *not* possible to translate full *Expr* with closures correctly.

The reason is that when a function value is produced, the values of its free variables are only known at runtime. But an executable sequence in *Pfx* is a static piece of code: without an extra mechanism, it cannot be modified so as to remember the current values of these free variables.

This is exactly the problem revealed in Question 10.4: the inner function is returned as plain code, not as a closure, so the value of *x* is lost when the first call finishes.

What must be added? We must add a new *Pfx* construct:

append

whose role is to transform a plain executable sequence into a real closure.

Idea of the translation. Let $\lambda x.e$ be a function, and let $FV(\lambda x.e) = \{z_1, \dots, z_k\}$ be its free variables in a fixed order.

- First, compile the body e as an executable sequence Q .
- Then, at runtime, when the values of the free variables are known, duplicate each value z_i from the stack.
- For each duplicated value, apply **append** to the executable sequence.

The result is a new executable sequence that begins by pushing the captured values of the free variables onto the stack before executing the original body. In other words, the executable sequence becomes a closure. So the answer is:

- no, not with the previous version of *Pfx*;
- yes, if we enrich *Pfx* with **append**, which allows closures to be constructed dynamically.

Question 13.2

We now give the operational semantics of **append**.

Stored command associated with a value. Let $\text{store}(v)$ be the instruction that pushes the value v on the stack when the closure is later executed:

$$\text{store}(n) = \text{push } n \quad \text{and} \quad \text{store}(\langle Q \rangle) = \langle Q \rangle.$$

Informal behavior. The instruction **append** expects:

- a value v on top of the stack;
- an executable sequence just below it.

It replaces these two stack elements by a new executable sequence that:

- first pushes v when the closure starts;
- then executes the old code;
- and finally removes the captured value from the stack when the function ends.

Formal rule. Using the same recursive interpretation of the paper notation $I.Q$ for putting a whole sequence in front of another, the small-step rule is:

$$\overline{(\text{append}.Q, v :: \langle Q' \rangle :: S) \longrightarrow (Q, \langle \text{store}(v).Q'.\text{swap.pop}.\emptyset \rangle :: S)}$$

Error cases. The machine raises a runtime error in all other cases:

- if the stack is empty;
- if the top element is missing;
- if the second element of the stack is not an executable sequence.

Why `swap.pop`? When the modified executable sequence terminates, its result is on top of the stack and the captured value is just below it. Therefore, removing the captured value while keeping the result requires `swap.pop` rather than only `pop`.